Nonexistence of Williamson matrices of order 65

P. Graham
https://orcid.org/0009-0007-8493-1505

October 24, 2026

Abstract

There are no Williamson matrices of order 65: four symmetric circulant sign matrices whose squares sum to 260 times the identity. Bright's March 20, 2019 research statement identifies this existence question. The computer-assisted argument reduces it by compression and symmetry, then exhaustively joins pairs of full rows. Separate checks establish coverage and accounting. The conclusion depends on correct execution of the exhaustive joins, which have not been formally verified.

1 The result

A *sign matrix* has entries in $\{1, -1\}$. A matrix is circulant when each row is the cyclic shift of its predecessor. Four symmetric circulant sign matrices A, B, C, D of order n are *Williamson matrices* if

$$A^2 + B^2 + C^2 + D^2 = 4nI_n, \tag{1}$$

where I_n is the identity matrix. We establish the following computer-assisted result.

Theorem 1. *There are no four symmetric circulant sign matrices A, B, C, D of order 65 satisfying $A^2 + B^2 + C^2 + D^2 = 260I_{65}$.*

This is a statement about symmetric circulant blocks, not about arbitrary Hadamard matrices of order 260 or the broader Williamson-type families. Order 67 remains open.

Computer-assisted; not yet reviewed by experts. The proofs, the search code, the certificate checkers and the text were produced by AI models (from OpenAI and Anthropic), working under the author's direction.

The existence question belongs to the Williamson construction of Hadamard matrices; see Bright, Kotsireas and Ganesh, arXiv version 2, Theorem 2 and Definition 5 [1]. Bright's research statement, dated March 20, 2019, Section 4.2 [2], calls 65 "currently the smallest order for which it is unknown" whether Williamson matrices exist. For the neighboring near-Williamson question, see Kharaghani, Mohammadian, Tayfeh-Rezaie and Zaitsev, arXiv version 2, Definition 1.1 and Table 6 [3].

This paper completes the order-65 nonexistence argument. Its ingredients include the compression and spectral conditions in Bright, Kotsireas and Ganesh, version 2, Definition 11, Theorem 12 and Corollaries 8–9, and Williamson's product theorem as stated there in Theorem 14 [1]. The difficult step here is excluding every full-row lift of every admissible compressed tuple: excluding compressed objects alone would not settle the question. A direct join of pairs of full rows completes that step, with conservative spectral bounds. The contribution is this complete exhaustion at order 65; no general construction principle or speedup over prior programs is claimed.

2 Proof overview

A symmetric row of order 65, normalized to start with +1, is encoded by its 32 remaining signs. Row sums and Fourier eigenvalues supply necessary conditions. Summing entries in each residue class modulo 13 compresses each row to an integer vector. Rows are grouped by row sum and compression, and four classes form an admissible tuple when their compressed autocorrelations satisfy the required equations. A lift chooses one full row from each class.

Common invertible multiplication of the indices and row permutation reduce these tuples to orbit representatives, called units. Every solution is equivalent to a lift of a unit. Multipliers fixing the compression allow a further restriction on one designated row. This restriction is applied to one row, not independently to all four.

Within each unit, the four lists are split into two pairs. Outward integer enclosures of squared eigenvalues give conservative spectral bounds: rounding cannot exclude a solution. Surviving pairs are joined using necessary autocorrelation and product identities, and matching candidates are tested exactly. Repeated classes and repeated rows are allowed. The appendix proves these reductions and explains why no solution can be lost. The exhaustive joins find none; the conclusion uses the execution assumptions in Section 3.

3 Checks, trust, and reproduction

This section distinguishes checking the exhaustion record from repeating the decisive computation. Two separately implemented coverage and accounting checkers regenerate the rows and complete compressed cover, check the receipts, and reject planted corruptions and cover gaps. The first reuses the CPU row generator but has separate cover code; the second has its own row, cosine, cover, and receipt code. Neither rederives all survivor or key-match counts, or proves that every pair was correctly filtered, stored, and matched. The receipts are an exhaustion record, not a short independently checkable proof of the unsuccessful joins. Independent here means separately implemented, not replication by separate human teams or institutions.

The theorem depends on the written reductions, the completeness of the row and cover enumerations, and correct execution of the exhaustive search. It also depends on the compiler, hardware, integer arithmetic, floating-point model, and interval calculations used for the numerical margins. Independent implementations, corruption rejection, and positive controls supply evidence against implementation errors, but the search has not been formally verified. No claim of a Lean proof or a short standard unsatisfiability certificate is made. A reader seeking to repeat the decisive computation must execute the full search, not only check its receipts.

From the result's root, use Python 3.10 or newer, GCC and G++ with OpenMP, and OpenSSL development headers and library:

```
python3 -m pip install -r checks/requirements.txt
python3 checks/check.py
```

The default check validates the record and runs known-case controls; it does not repeat every order-65 join. Add `-full-controls` for the longer known-case controls or `-full-search` for the complete CPU exhaustion. The numerical and storage arguments follow in the appendix; the exact command scope, historical CPU and GPU receipts, sample checks, and their limitations are in `checks/README.md` and `checks/computation.md`. The large row data are regenerated; no external dataset is required and no separate dataset DOI exists.

A Rows, autocorrelations, and spectra

This section fixes the encoding and proves the necessary conditions used to retain individual rows. All indices below are taken modulo the odd order n, and $H = (n-1)/2$.

For example, at order 5 a normalized symmetric first row has the form $(1, a, b, b, a)$, with $a, b \in \{1, -1\}$. In general a normalized row is a sequence $x = (x_i)_{i \in \mathbb{Z}/n\mathbb{Z}}$ with $x_0 = 1$ and $x_i = x_{-i}$. Encode it by the H-bit integer m_x whose bit $i-1$ is one exactly when $x_i = -1$, for $1 \le i \le H$. The function $\mathrm{popcount}(m_x)$ counts the one bits of this mask. Define its periodic autocorrelation by

$$\mathrm{PAF}_x(s) = \sum_{i \in \mathbb{Z}/n\mathbb{Z}} x_i x_{i+s}.$$

For a symmetric circulant matrix with first row x, the entry in position $(0, s)$ of its square is $\mathrm{PAF}_x(s)$. Thus the matrix equation is exactly

$$\mathrm{PAF}_a(s) + \mathrm{PAF}_b(s) + \mathrm{PAF}_c(s) + \mathrm{PAF}_d(s) = \begin{cases} 4n, & s = 0, \\ 0, & s \neq 0. \end{cases} \tag{2}$$

Whenever a sum below runs over the four rows, it counts their four positions, including repetitions. Autocorrelation is symmetric in s, so testing $0 \le s \le H$ suffices.

Lemma 2. *If Williamson matrices of odd order n exist, their first rows may be normalized to $x_0 = 1$. Each normalized row has row sum $r_x = n - 4\,\mathrm{popcount}(m_x)$, and the four row sums satisfy $r_a^2 + r_b^2 + r_c^2 + r_d^2 = 4n$.*

Proof. Negation preserves every autocorrelation. Negative entries in a normalized symmetric row occur in pairs, giving the stated row sum. Summing (2) over all shifts gives the sum of the squares of the row sums. □

At order 65 the nondecreasing lists of absolute row sums are exactly

$$(1, 3, 5, 15),\ (1, 3, 9, 13),\ (3, 3, 11, 11),\ (3, 7, 9, 11),\ (7, 7, 9, 9).$$

Each sign is forced by $r_x \equiv 65 \pmod 4$, giving the allowed signed values $1, -3, 5, -7, 9, -11, 13, -15$. Permuting the four rows therefore permits sorting by the absolute row sums without loss.

For frequency $k \in \mathbb{Z}/n\mathbb{Z}$, the real eigenvalue of the circulant is

$$\lambda_x(k) = 1 + 2\sum_{i=1}^{H} x_i \cos(2\pi i k/n).$$

The Fourier transform diagonalizes all circulant matrices. Consequently (2) is equivalent to

$$\lambda_a(k)^2 + \lambda_b(k)^2 + \lambda_c(k)^2 + \lambda_d(k)^2 = 4n \qquad (k \in \mathbb{Z}/n\mathbb{Z}). \tag{3}$$

In particular every solution row satisfies $\lambda_x(k)^2 \le 4n$. The row generator considers every mask and retains only permitted row sums and rows satisfying this necessary spectral bound, with the numerical slack justified in Section C.1.

B Compression and complete coverage

We now divide the possible solutions into finitely many units and justify both symmetry reductions. These are reductions of the search domain; they do not assert that a compressed solution necessarily lifts.

For $n = qL$, with compression factor q, define the compression of a row x by

$$X_j = \sum_{t=0}^{q-1} x_{j+Lt}, \qquad j \in \mathbb{Z}/L\mathbb{Z}.$$

Here $q = 5$ and $L = 13$. A class consists of the retained rows with the same signed row sum and compressed vector. Write $\mathrm{PAF}_X(s) = \sum_{j \in \mathbb{Z}/L\mathbb{Z}} X_j X_{j+s}$ for compressed autocorrelation.

Lemma 3. *For four rows satisfying (2), their compressions satisfy*

$$\mathrm{PAF}_A(s) + \mathrm{PAF}_B(s) + \mathrm{PAF}_C(s) + \mathrm{PAF}_D(s) = \begin{cases} 4n, & s = 0 \pmod L, \\ 0, & s \neq 0 \pmod L. \end{cases}$$

Proof. Expanding the compressed autocorrelation gives $\mathrm{PAF}_X(s) = \sum_{t=0}^{q-1} \mathrm{PAF}_x(s + Lt)$. For a nonzero residue s none of these shifts is zero modulo n. For $s = 0$ exactly one is zero. Summing over the four rows proves the claim. □

Call a tuple of four classes *admissible* when its compressed vectors satisfy Lemma 3 and its row sums satisfy Lemma 2. A *lift* is a tuple of four retained rows, one from each class; it need not itself solve the matrix equation. Classes are ordered lexicographically by the seven entries $(X_0, \ldots, X_6)$ of their symmetric compression. A class identifier is its zero-based rank in this order; the data files use those identifiers. A *sorted* class tuple has nondecreasing absolute row sums and, within equal row sums, nondecreasing class identifiers. Acting by a multiplier means applying it to each compressed vector and then re-sorting within equal-row-sum positions. The generator and cover check use this same order.

The admissible class tuples are enumerated by matching sums of two exact compressed autocorrelation vectors to the complementary pair sums. Every ordered class pair is included; equal classes are permitted. Sorting positions of equal absolute row sum removes only row permutations. The complete tuple set is also enumerated independently by matching the exact integer vectors themselves.

A *multiplier* is an invertible residue u modulo 65, acting on a row by $x_i \mapsto x_{ui}$. It preserves (2), since it permutes the shifts. Its action on the compression depends only on its residue modulo 13. Opposite multipliers act identically on symmetric rows. Every invertible residue modulo 13 lifts to an invertible residue modulo 65: among $u, u + 13, \ldots, u + 52$, exactly one is divisible by 5, so the other four are coprime to 65.

Lemma 4. *Let one representative of each orbit of admissible sorted class tuples under* $(\mathbb{Z}/13\mathbb{Z})^*/\{1, -1\}$ *be retained. Every order-*65 *Williamson solution is equivalent, by a common multiplier and a permutation of its rows, to a lift of a retained tuple.*

Proof. A solution's tuple is admissible by Lemma 3. Choose a multiplier carrying it to its orbit representative, lift that multiplier modulo 65, and apply it to all four full rows. Re-sort rows of equal absolute row sum. The resulting solution has the representative tuple. All lifts of the representative are searched, even if more than one multiplier carries the original tuple to it. Multipliers permute the exact eigenvalues. Transformed solution rows therefore still satisfy $\lambda_x(k)^2 \leq 260$ and pass the numerical filter with its proved margin. □

A *unit* is one representative in this lemma. The cover check computes the exact compressed tuples and verifies membership of their representatives in the supplied unit list. It checks invalid and duplicate representatives as well as missing ones.

A further symmetry reduces one row list in each unit. The subgroup

$$K = \{u \in (\mathbb{Z}/65\mathbb{Z})^* : u \equiv 1 \pmod{13}\} = \{1, 14, 27, 53\}$$

fixes every compressed vector. Choose one designated position in the unit and retain only rows whose mask is least in its K-orbit. Applying a common element of K that minimizes that row preserves a solution in the same unit. It is essential that this restriction applies to only one designated row; independent minimization of all four rows would not be justified.

C Filtering and matching full rows

This section shows that every solution lifting a unit survives the pair filters and is reached by the join. A pairing is one of the three splits $(a, b) \mid (c, d)$, $(a, c) \mid (b, d)$, or $(a, d) \mid (b, c)$. The implementation may choose any of these pairings and any designated row; these choices affect cost only.

C.1 Conservative numerical enclosures

We use integer enclosures to make the spectral comparisons safe under rounding. At order 65, the error in each computed squared eigenvalue is below $4 \cdot 10^{-11}$ under the arithmetic assumptions and cosine-table checks detailed in Appendix E. The row filter rejects only when the computed square exceeds $260 + 10^{-6}$; it therefore retains every solution row.

Let $\widetilde{\lambda}_x(k)$ be the computed eigenvalue. Define integer lower and upper enclosures

$$\ell_x(k) = \max\{0, \lfloor 64\widetilde{\lambda}_x(k)^2 - 0.01 \rfloor\}, \qquad h_x(k) = \lceil 64\widetilde{\lambda}_x(k)^2 + 0.01 \rceil.$$

The error bound, including the two additional floating-point additions, implies

$$\ell_x(k) \le 64\lambda_x(k)^2 \le h_x(k). \tag{4}$$

Clamping the lower bound to zero preserves this enclosure, since a square is nonnegative. No strictly positive gap is claimed after clamping.

C.2 The pair filter

We now use the opposite pair of row lists to bound the possible spectral contribution of each pair. Consider two lists $\mathcal{A}, \mathcal{B}$ on one side and $\mathcal{C}, \mathcal{D}$ on the other, after restricting the designated row. Set $Q = 64 \cdot 260$. For each frequency let

$$U(k) = \max\{0, Q - \min_{c \in \mathcal{C}} \ell_c(k) - \min_{d \in \mathcal{D}} \ell_d(k)\},$$

$$V(k) = \max\{0, Q - \max_{c \in \mathcal{C}} h_c(k) - \max_{d \in \mathcal{D}} h_d(k)\}.$$

Retain a pair (a, b) only when, at every frequency,

$$\ell_a(k) + \ell_b(k) \le U(k), \qquad h_a(k) + h_b(k) \ge V(k). \tag{5}$$

The opposite side uses the same rule with the two sides interchanged.

Lemma 5. *Every pair belonging to a solution in the four lists passes* (5) *on both sides.*

Proof. For its complementary rows c, d, equation (3) gives $64(\lambda_a(k)^2 + \lambda_b(k)^2) = Q - 64\lambda_c(k)^2 - 64\lambda_d(k)^2$. Replacing each square by the appropriate lower or upper enclosure proves the inequalities before clamping. Raising a negative upper bound to zero only weakens the upper test. Raising a negative lower bound to zero changes nothing, because $h_a + h_b$ is nonnegative. □

If a list is empty there is no solution in the unit. The implementation details are given in Appendix F.

C.3 The product identity and the join

The second matching condition is Williamson's classical product identity [1, version 2, Theorem 14]. We include its short argument to fix the normalization.

Lemma 6. *For an odd-order Williamson solution normalized by $x_0 = 1$ in each row, $a_i b_i c_i d_i = -1$ for every nonzero index i. Equivalently, $m_a \oplus m_b \oplus m_c \oplus m_d = 2^H - 1$, where $\oplus$ is bitwise exclusive-or on H bits.*

Proof. In the integral group ring of the cyclic group of order n, write J for the sum of all group elements and T_x for the sum of the positions where row x is negative. Then $x = J - 2T_x$ and $x^2 = nJ - 4|T_x|J + 4T_x^2$. Symmetry and normalization make $|T_x|$ even. Modulo 2, the square T_x^2 equals the sum of the squares of its individual group elements, because mixed terms occur twice. Reducing $\sum_x x^2 = 4n\,1$ modulo 8 and dividing by 4 modulo 2 gives

$$\sum_x \sum_{t \in T_x} t^2 = 1 + J \pmod 2.$$

Squaring permutes a cyclic group of odd order. Every nonidentity position therefore occurs in an odd number of the four negative sets, proving the product identity and its mask form. □

For each row, form a linear hash of its nonzero-shift integer autocorrelations,

$$f(x) = \sum_{s=1}^{H} R_s \,\mathrm{PAF}_x(s) \pmod{2^{64}},$$

using fixed odd coefficients R_s; their reproducible choice is described in Appendix F. A solution satisfies $f(a) + f(b) = -f(c) - f(d)$ and $m_a \oplus m_b = (2^H - 1) \oplus m_c \oplus m_d$. The search combines these two necessary equalities into a single 64-bit key, using the same function on both sides. Specifically it uses $\alpha \oplus R\beta$ for hash sum α and mask value β, with a fixed odd R, and maps a zero key to one. The right side uses the negative hash sum and the H-bit complement of its mask XOR. Equal inputs give equal keys; hash collisions may add candidates but cannot remove a solution.

The join retains every pair with an equal key, including duplicates. Its storage and restart rules, justified in Appendix F, do not remove any such pair.

Finally, each key match is checked using the four exact integer autocorrelation vectors at shifts $0, \ldots, H$. Equation (2) then decides the matrix equation exactly. A separate witness checker can also form the four matrices and multiply them directly. This last check prevents false positives; it cannot repair a missing pair earlier in the search.

D Exhaustion and conclusion

We state the finite computational result needed to finish the proof. The CPU search regenerates its inputs from the problem definition. Its counts are shown in Table 1. A receipt is a record for one unit: the four classes, pairing, designated row, list sizes, enumerated pair counts, survivor counts, matching keys, solutions, and number of passes. A pass processes one part of the pair lists, as defined in Appendix F; every order-65 unit uses one pass. A key match is a surviving left pair and a surviving right pair with equal keys. A surviving pair is counted once for each side of each unit where it occurs. The complete receipt collection has no unresolved unit.

Quantity	Count
Normalized masks examined	2^{32}
Retained rows	396,050,997
Compressed classes	15,251
Admissible sorted class tuples	423,527
Units after multiplier reduction	70,699
Enumerated pairs, both sides	70,830,746,132,488
Surviving pairs, both sides	1,645,891,558,124
Matches between pair keys	0
Solutions	0

Table 1: Counts in the complete CPU exhaustion record at order 65.

Computer-assisted conclusion of Theorem 1. Every solution may be normalized and belongs to the retained row set by Lemma 2 and the proved spectral margin. Lemmas 3 and 4 place an equivalent solution in one of the covered units. The restriction on the designated row preserves a solution in that unit. Lemmas 5 and 6 show that its two pairs survive and have complementary mask values. Moreover, $f(a) + f(b) + f(c) + f(d) = \sum_{s=1}^{H} R_s(\mathrm{PAF}_a(s) + \mathrm{PAF}_b(s) + \mathrm{PAF}_c(s) + \mathrm{PAF}_d(s)) = 0$ modulo 2^{64}, so their join keys are equal. The exhaustive joins report no key match in any unit. Subject to the search-execution trust base in Section 3, no solution exists. □

E Numerical error bound

This appendix supplies the arithmetic details behind the conservative spectral enclosures used in Section C.1.

The row generator and the search evaluate the cosine table in IEEE binary64 arithmetic, rounded to nearest, without fast-math transformations. The public build invokes GCC with `-O3 -fopenmp` and links `libm`; it invokes G++ with `-O3 -std=c++17 -fopenmp` for the second checker. The recorded CPU search additionally used `-march=native`. No build uses `-ffast-math`. The numerical bound assumes ordinary IEEE binary64 operations rather than x87 extended intermediate precision; a build changing the arithmetic model requires its own justification. The actual table entries are checked against interval evaluations of cosine at 50 decimal digits. Their errors are at most 2^{-49}. For order 65 the eigenvalue sum is split into two sums of at most 16 terms. Each partial sum has magnitude at most 16, so the error in one half is bounded by

$$16\,2^{-49} + 16^2\,2^{-53} < 6 \cdot 10^{-14}.$$

Combining the halves, doubling, and adding one gives eigenvalue error below $2.6 \cdot 10^{-13}$. Since $|\lambda_x(k)| \le 65$, the error in the computed square, including rounding of the multiplication, is below $4 \cdot 10^{-11}$. Fusing a multiplication and addition does not enlarge this bound. The row filter rejects only when the computed square exceeds $260 + 10^{-6}$, so it retains every solution row.

F Storage and key implementation

This appendix explains why the implemented key and data structures preserve every pair equality required by a solution. The coefficients R_s are the successive outputs of the public `splitmix64` function, initialized with the hexadecimal seed `5713a1d2c0ffee65`, with the low bit set to one. The outputs are indexed from $s = 0$ to 32, with indices 1 through 32 used in f. The fixed multiplier R is hexadecimal `d6e8feb86659fd93`. The complete key function is specified in `checks/search.c`. Their values affect collisions and cost, but the argument needs only that both sides use the same function.

The pair bounds are formed in signed integer arithmetic before conversion to unsigned 16-bit values. The stored row values are below 16,642; all intervening sums and differences fit the signed integer type. Frequency reordering and vectorized comparisons do not change the predicates. In a vectorized upper test, a row with $\ell_x > U$ is discarded first, before forming $U - \ell_x$; the lower difference is $\max\{0, V - h_x\}$. The remaining frequencies are tested with signed integer sums. Empty lists contain no pairs and cannot yield a solution.

Surviving pairs on one side are stored by linear probing in a table whose size exceeds twice the number of entries. No entry is deleted or overwritten. A probe continues through every equal key until the first empty slot, so repeated keys and repeated rows are retained. A capacity overflow abandons the whole attempt and restarts in passes partitioned by the high bits of this same key on both sides. Equal keys always belong to the same pass. Every reported order-65 unit completed in one pass.

References

[1] C. Bright, I. Kotsireas and V. Ganesh, Applying Computer Algebra Systems with SAT Solvers to the Williamson Conjecture, arXiv:1804.01172v2, May 14, 2019; first posted April 3, 2018; Journal of Symbolic Computation **100** (2020), 187–209. https://arxiv.org/abs/1804.01172v2. https://doi.org/10.1016/j.jsc.2019.07.024.

[2] C. Bright, Extended Research Statement, March 20, 2019. https://cs.curtisbright.com/extended-research-statement.pdf.

[3] H. Kharaghani, A. Mohammadian, B. Tayfeh-Rezaie and V. Zaitsev, A Search for Hadamard Matrices of Near-Williamson Type, arXiv:2605.08661v2, September 6, 2026. First posted May 9, 2026 as version 1 by H. Kharaghani, A. Mohammadian and B. Tayfeh-Rezaie, titled *A search for Hadamard matrices of Williamson type*. https://arxiv.org/html/2605.08661v2.